Cloud Computing Course

Course n°4: CI/CD for the cloud

Maximilien Walbecq
Data Engineer
Roquette Frères 🌱

Quiz !

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

1. Which of the following best describes a container?

A. A full virtualized system including its own OS
B. A lightweight package that includes code and its dependencies
C. A cloud-managed serverless function
D. A storage unit for large unstructured data

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

1. Which of the following best describes a container?

A. A full virtualized system including its own OS
B. A lightweight package that includes code and its dependencies
C. A cloud-managed serverless function
D. A storage unit for large unstructured data

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

2. What is one main advantage of PaaS compared to IaaS?

A. No need to manage servers or infrastructure
B. You have full control over the operating system
C. You must manage scaling manually
D. You don't need to configure anything

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

2. What is one main advantage of PaaS compared to IaaS?

A. No need to manage servers or infrastructure
B. You have full control over the operating system
C. You must manage scaling manually
D. You don't need to configure anything

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

3. What is the purpose of a Dockerfile?

A. To describe how to build a container image
B. To store environment variables
C. To define user permissions inside the container
D. To monitor running containers

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

3. What is the purpose of a Dockerfile?

A. To describe how to build a container image
B. To store environment variables
C. To define user permissions inside the container
D. To monitor running containers

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

4. In the Twelve-Factor App methodology, what does the “Config” factor recommend?

A. Hardcode all configurations for better control
B. Declare dependencies explicitly (e.g., in **requirements.txt**)
C. Store configuration in environment variables
D. A single codebase per app, stored in a version control system

© Maximilien WALBECQ - Created with Marp e MarpX.

Quiz

4. In the Twelve-Factor App methodology, what does the “Config” factor recommend?

A. Hardcode all configurations for better control
B. Declare dependencies explicitly (e.g., in **requirements.txt**)
C. Store configuration in environment variables
D. A single codebase per app, stored in a version control system

© Maximilien WALBECQ - Created with Marp e MarpX.

Reminder of previous course

Application on the cloud

- Explore the compute resources from the last session in more detail
- Discover the PaaS solutions
- Try to deploy applications on the cloud

© Maximilien WALBECQ - Created with Marp e MarpX.

Contenu

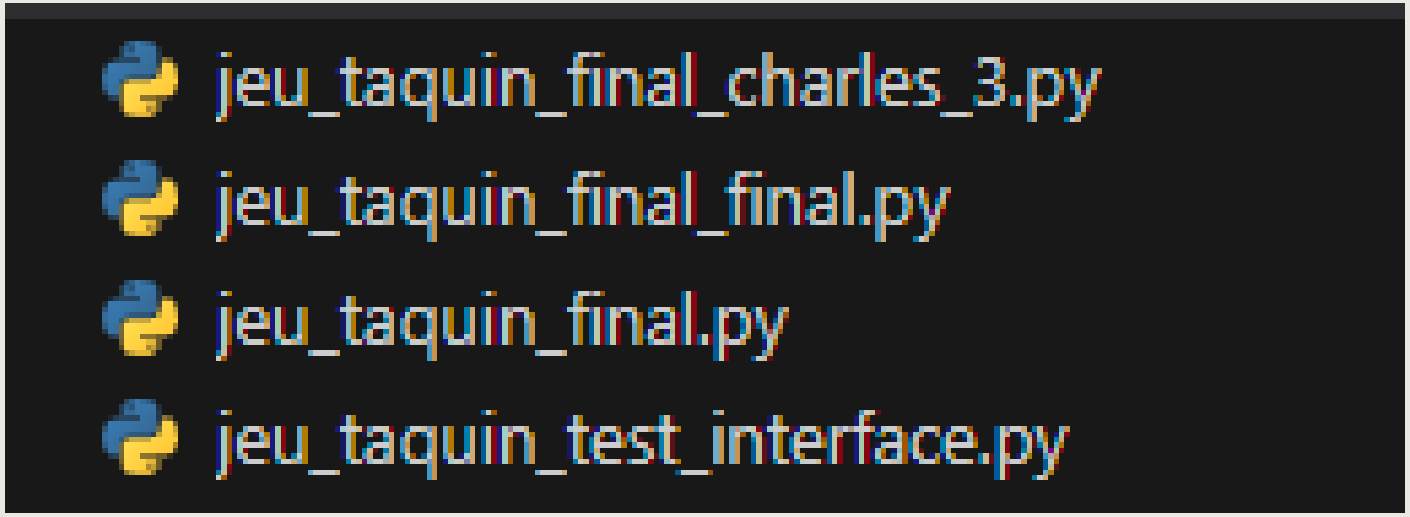
CI/CD for the cloud

1. Title
2. Reminder of previous courses
3. What is git
4. Git Branching Strategies
5. What is CI/CD

© Maximilien WALBECQ - Created with Marp e MarpX.

Git Fundamentals

First time coding



© Maximilien WALBECQ - Created with Marp e MarpX.

What is git

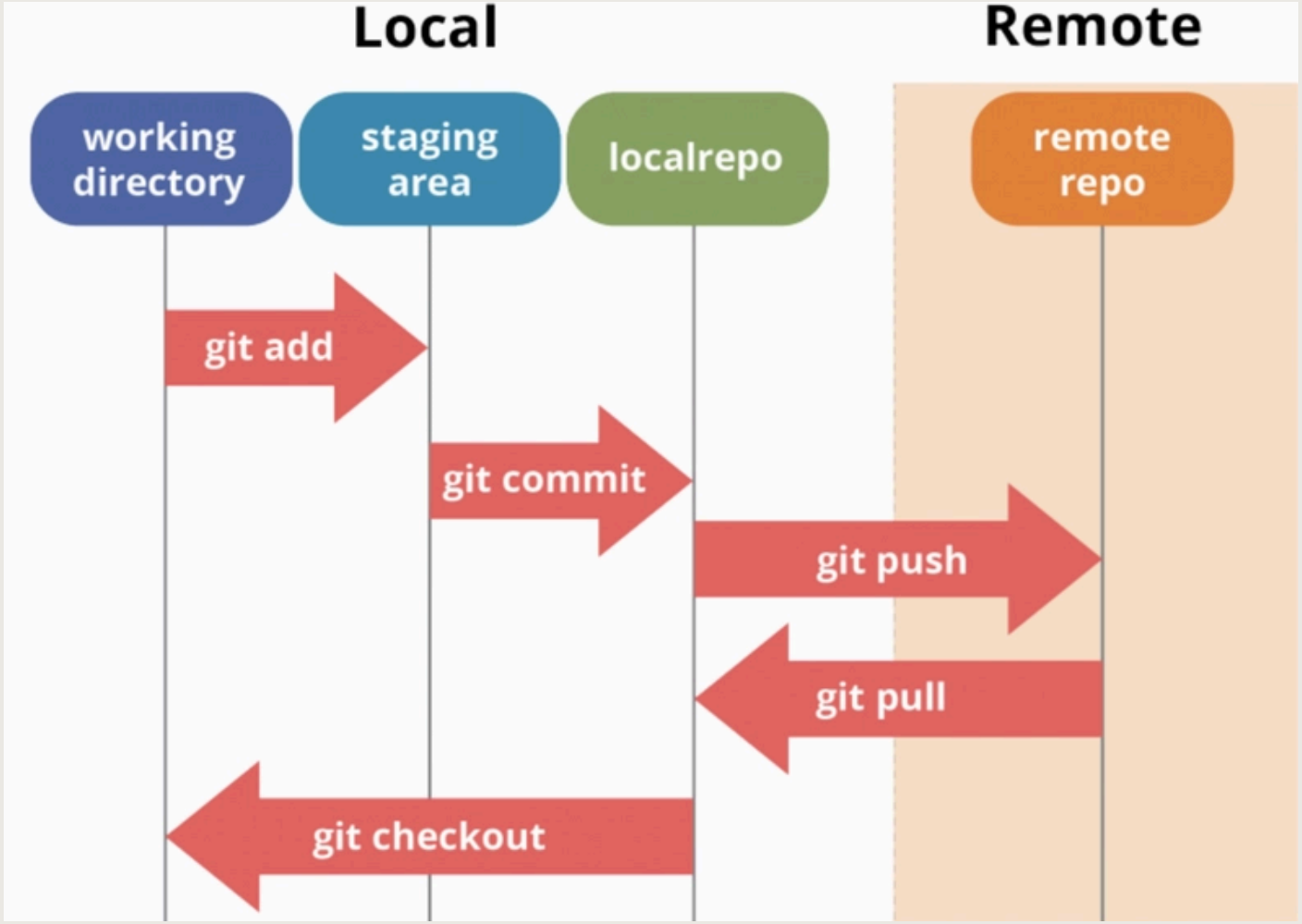
Distributed Version Control System

- Track changes in source code
- Collaborate with multiple developers
- Maintain complete project history
- Develop and integrate new features effectively



© Maximilien WALBECQ - Created with Marp e MarpX.

Git Workflow



© Maximilien WALBECQ - Created with Marp e MarpX.

Essential Git Commands

```
git init                    # Create new repository
git clone <url>             # Clone remote repository

git add <file>              # Stage changes
git commit -m "message"     # Commit changes
git push                    # Push to remote
git pull                    # Pull from remote
```

Note

You can also do most of these actions through the interface on Visual Studio Code !

© Maximilien WALBECQ - Created with Marp e MarpX.

Essential Git Commands

```
git branch <name>           # Create branch
git checkout <name>         # Switch branch
git checkout -b <name>      # Create and switch

git switch <name>           # Switch branch
git switch -c <name>        # Create and switch

git merge <branch>          # Merge branch
git branch -d <branch>      # Delete branch
```


© Maximilien WALBECQ - Created with Marp e MarpX.

Git Branching Strategies

Git Branching Strategies

Git Flow

Main Branches:

- **main**: Contains production-ready code
- **develop**: Integration branch where new features are merged

Supporting Branches:

- **feature/**: Building branch where new features are developed
- **release/**: Release preparation before merge into main
- **hotfix/**: Urgent production fixes branch.

© Maximilien WALBECQ - Created with Marp e MarpX.

Git Branching Strategies



© Maximilien WALBECQ - Created with Marp e MarpX.

Git Branching Strategies

Git Flow

When to use

- Great for teams with regular development and release
- Complex release management or compliance requirements
- Multiple versions in production

Disadvantages

- Important complexity, not ideal for smaller teams
- Not great for continuous deployment
- Complex Branch management

© Maximilien WALBECQ - Created with Marp e MarpX.

Git Branching Strategies

Git Hub Flow

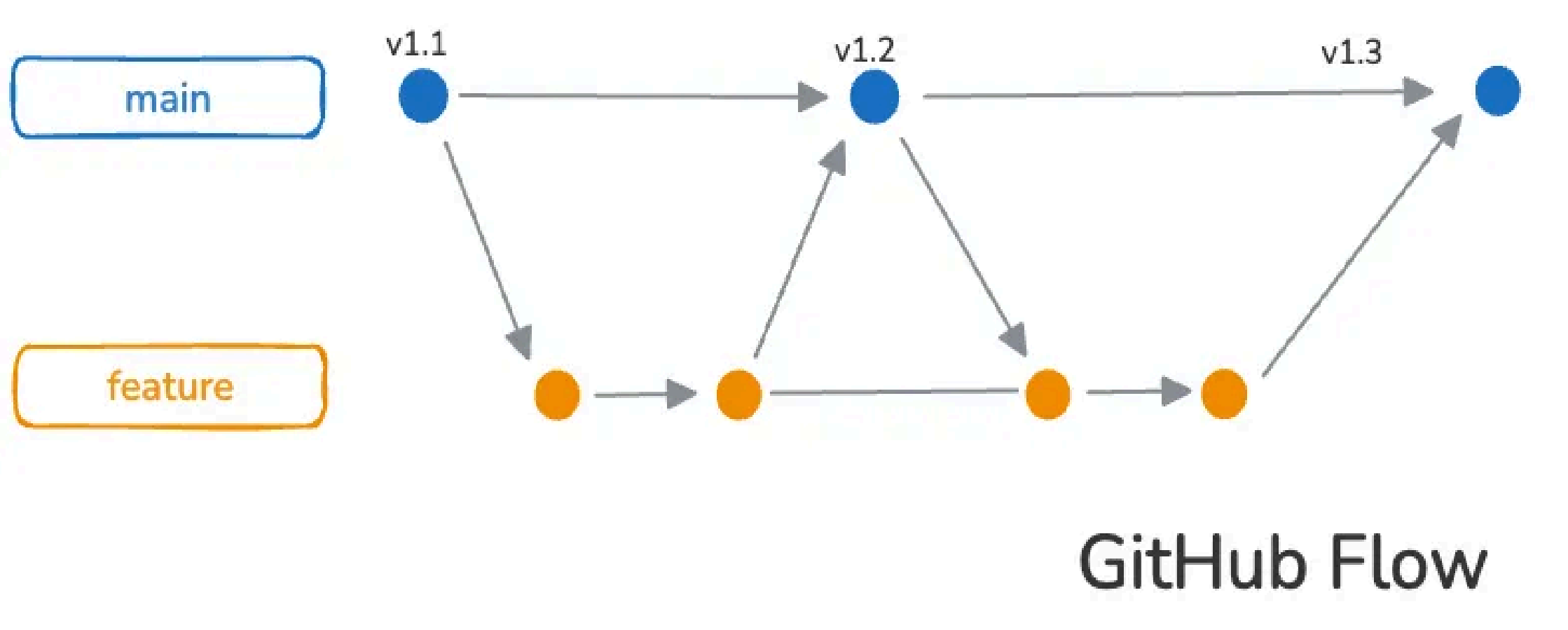
- main branch is always deployable
- Create feature branches from main
- Open pull request for review
- Merge to main after approval
- Deploy immediately

Trunk-Based Development

- Almost the same process
- Short-lived feature branches (< 1 day)
- Feature flags for incomplete features

© Maximilien WALBECQ - Created with Marp e MarpX.

Git Branching Strategies



© Maximilien WALBECQ - Created with Marp e MarpX.

Git Branching Strategies


center

© Maximilien WALBECQ - Created with Marp e MarpX.

Git Branching Strategies

GHF / TBD

When to use

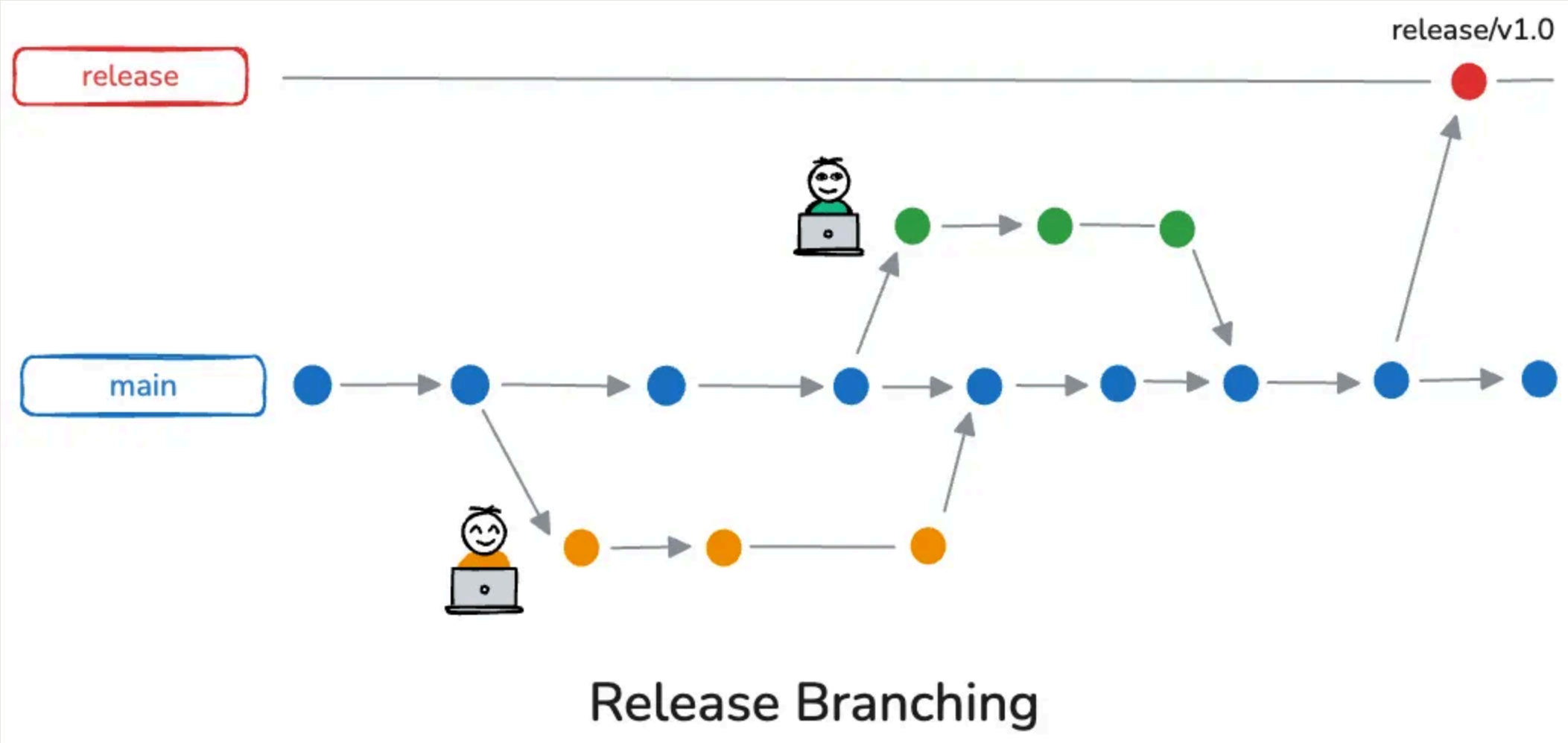
- For continuous deployment
- When you have strong automated testing
- Frequent & small releases
- Small team

Disadvantages

- Can be more complex with important teams
- Requires to rigorous with git practices

© Maximilien WALBECQ - Created with Marp e MarpX.

Git Branching Strategies



© Maximilien WALBECQ - Created with Marp e MarpX.

Git best practices

Commit Best Practices

Best pratices

- Small, focused, frequent commits
- Write clear messages
- Don't mix features and fixes
- Don't mix linter with your team

Examples

- feat: New feature
- fix: Bug fix
- docs: Documentation
- test: Adding tests

© Maximilien WALBECQ - Created with Marp e MarpX.

Use a .gitignore file

```
# Build outputs
dist/
build/
*.exe

# Environment files
.env
.env.local

# IDE
.vscode/
.idea/
*.swp

# Logs
*.log
logs/
```


© Maximilien WALBECQ - Created with Marp e MarpX.

Continuous Integration

and Continuous Deployment

What is CI/CD

Continuous Integration (CI)

- Automated building and testing of code changes
- Frequent integration into shared repository
- Early bug detection

Continuous Deployment (CD)

- Automated deployment to production
- Rapid release cycles
- Minimal manual intervention

© Maximilien WALBECQ - Created with Marp e MarpX.

Why Git Matters for CI/CD

- Single Source of Truth: All code in one place
- Trigger Automation: Commits trigger pipelines
- Collaboration: Team works on same codebase easily
- Rollback Capability: Easy to revert changes
- Branching Strategy: Organized development workflow

© Maximilien WALBECQ - Created with Marp e MarpX.

Tools

Azure DevOps

Components:

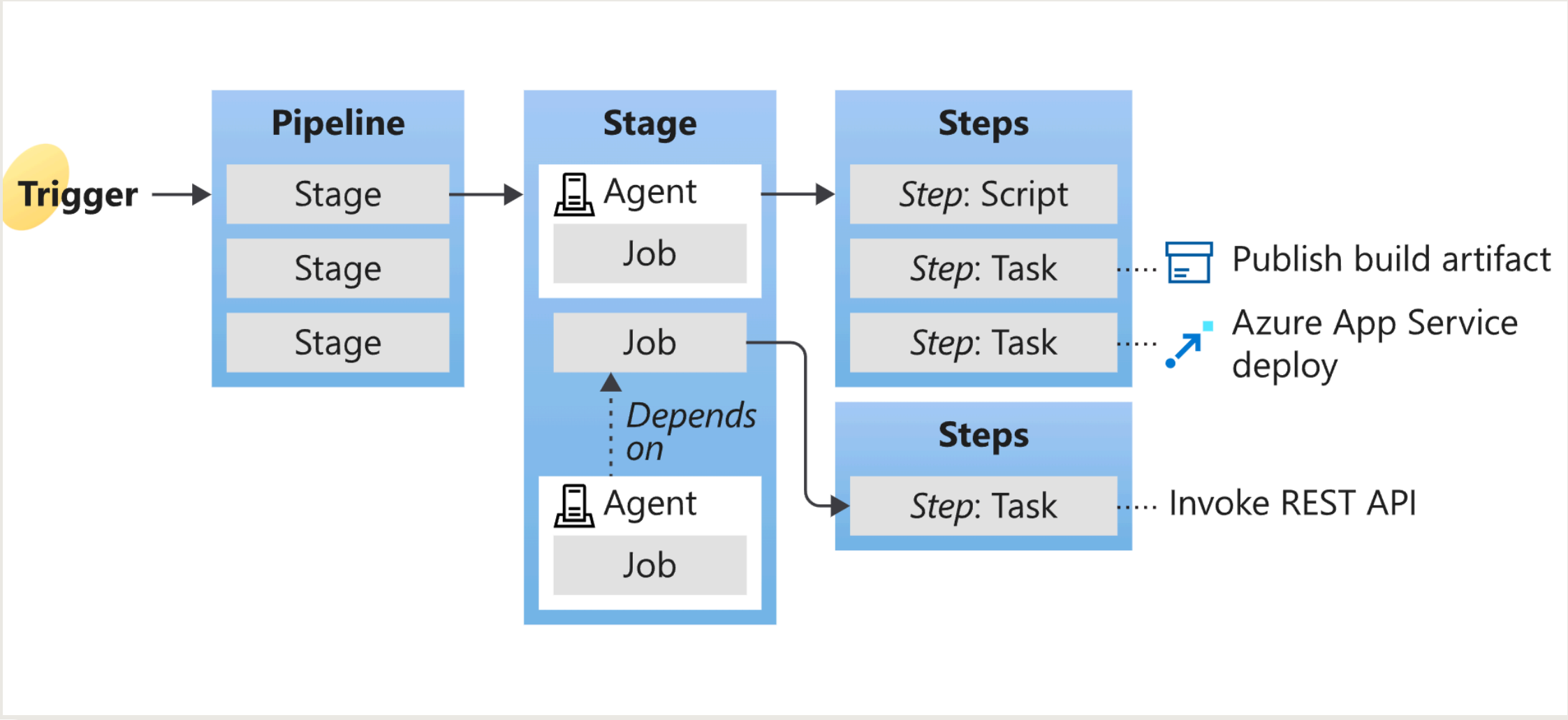
- Azure Pipelines - CI/CD
- Azure Repos - Git hosting

Others

- Github Actions
- Gitlab CI/CD
- Other cloud providers tools (AWS, GCP)

© Maximilien WALBECQ - Created with Marp e MarpX.

Azure Devops



© Maximilien WAILBEGO - Created with Marp & MarpX

https://learn.microsoft.com/en-us/azure/devops/pipelines/get-started/key-pipelines-

CI/CD

How to implement CI ?

- Set up a version control system like Git.
- Choose a git branching strategy.
- Use feature branches for new code and merge only after review.
- Add automated tests (unit, integration) to validate every commit.
- Configure a CI pipeline using tools like Azure Pipeline or GitHub Actions.
- Trigger builds automatically on every push or pull request.

© Maximilien WALBECQ - Created with Marp e MarpX.

Unit Tests

Definition

- Unit tests test individual units (modules, functions, classes) in isolation from the rest of the program. (definition from TDD)
- Written and run by developers to catch bugs (early or too late)
- Use mocks/fixture to simulate dependencies
- Help ensure refactoring and changes don’t break core logic

© Maximilien WALBECQ - Created with Marp e MarpX.

Integration Tests

Definition

- Verify that multiple components or modules work together as expected
- Check the integration between a database, API, external services

© Maximilien WALBECQ - Created with Marp e MarpX.

Example

You can use pytest or unittest

```
import unittest

def addition(a, b):
    return a + b

class TestMathOperations(unittest.TestCase):
    def test_addition(self):
        self.assertEqual(addition(2, 3), 5)
        self.assertEqual(addition(-1, 1), 0)

if __name__ == '__main__':
    unittest.main()
```


© Maximilien WALBECQ - Created with Marp e MarpX.

Example

Terraform

- terraform validate: check your file syntax
- terraform plan: generate an execution plan
  - For multiple person project, the state file can be shared on a cloud storage

You can also use tools for static code analysis to detect security problems and compliance inconsistency!

See more

- https://learn.microsoft.com/en-us/azure/developer/terraform/best-practices-integration-testing

© Maximilien WALBECQ - Created with Marp e MarpX.

CI/CD

How to implement CD ?

- Extend the CI pipeline to include deployment steps.
- Automate packaging and deployment (e.g., Docker, Kubernetes, or a cloud service).
- Deploy to environment stages (dev, qua, prod)
- Run automated tests before promoting code to the next stage.
- Require approval or automatic rules for production deployment.

© Maximilien WALBECQ - Created with Marp e MarpX.

Reminder: The Twelve Factors App

1. Codebase

One codebase tracked in revision control, many deploys

- A single codebase per app, stored in a version control system.
- Multiple deploys (production, staging, etc.) share the same codebase.
- Different apps should have different repositories.

© Maximilien WALBECQ - Created with Marp e MarpX.

Thank you !

Let's take a break before the practice session!